

GRADO EN INGENIERÍA INFORMÁTICA
DESARROLLO DE SOFTWARE



**UNIVERSIDAD
DE GRANADA**

P3: Sistema Bancario con Pruebas Unitarias en Flutter/Dart

Blanca Girón Ricoy (*blancagiron@correo.ugr.es*)

Karim Said Lupiañez (*karimsaidl@correo.ugr.es*)

Pablo Tamayo López (*pablotl0@correo.ugr.es*)

Repositorio: <https://github.com/blancagiron/Desarrollo-Software>

Marzo 2025

Índice

1	Introducción y objetivos	3
1.1	Introducción	3
1.2	Objetivos de la práctica	3
2	Implementación en Dart	4
2.1	Proceso de transformación, UML a código	4
2.1.1	Clase Account	4
2.1.2	Clase Transaction	5
2.1.3	Clase BankService	5
2.2	Estructura de ficheros	6
3	Diseño e implementación de pruebas	6
3.1	Enfoque de Pruebas	6
3.2	Herramientas utilizadas	7
3.3	Agrupaciones de tests	7
3.3.1	Grupo Account	7
3.3.2	Grupo Transaction	8
3.3.3	Grupo BankService	10
4	Diseño de la interfaz	12
4.1	Control de errores y validaciones	13
4.2	Interacción	13
4.3	Resultado final	13

1. Introducción y objetivos

1.1. Introducción

En el proceso de desarrollo de software, ser capaces de comprender, analizar y transformar diseños como los diagramas UML en implementaciones concretas es esencial, ya que estos diagramas no solo son una herramienta de documentación, sino que permiten planificar arquitecturas software robustas, escalables y fáciles de mantener. En esta práctica, se desarrolla un sistema bancario utilizando el lenguaje **Dart**, el cual forma parte del ecosistema de **Flutter**, partiendo del análisis de un diagrama de clases UML que describe una arquitectura modular, compuesta por clases como `Account`, `Transaction` (y sus subclases) y `BankService`, que actúa como fachada. El objetivo principal es transformar el diseño en una solución funcional, capaz de registrar cuentas, ejecutar distintas operaciones como retirar o transferir, y mantener un historial coherente de transacciones con identificadores únicos.

Además del diseño orientado a objetos, esta práctica enfatiza la importancia del **diseño de pruebas unitarias**. La creación de estas pruebas no debe limitarse únicamente a verificar que el código funciona correctamente, sino que debe buscar de forma activa condiciones que puedan causar errores y comprobar el comportamiento del código ante distintas situaciones. Diseñar casos de prueba para estas situaciones, normales, límites y excepcionales, permite detectar un mayor número de errores, reducir fallos en el desarrollo y mejorar la fiabilidad del sistema.

Para ello, se hace uso del paquete `test` de Dart, que permite estructurar pruebas mediante funciones como `group()` y `test()`, proporcionando una herramienta útil para validar el comportamiento que se espera de cada clase. Se implementarán pruebas específicas para las distintas clases, abarcando tanto flujos positivos como negativos.

1.2. Objetivos de la práctica

Los objetivos principales de esta práctica son:

- Comprender y analizar un diagrama de clases UML, identificando las relaciones entre las entidades, atributos clave y métodos que definen el comportamiento esperado del sistema.
- Ser capaces de diseñar software orientado a objetos en Dart a partir de dicho análisis, respetando principios clave de encapsulamiento, herencia y abstracción.
- Desarrollar un conjunto de clases que permitan simular un sistema bancario.
- Diseñar pruebas unitarias que maximicen la detección de errores.
- Familiarizarse con los conceptos de `group()` y `test()`, y comprender su utilidad para organizar pruebas en torno a funcionalidades o clases específicas.

2. Implementación en Dart

2.1. Proceso de transformación, UML a código

El diagrama UML propuesto en clase es el siguiente:

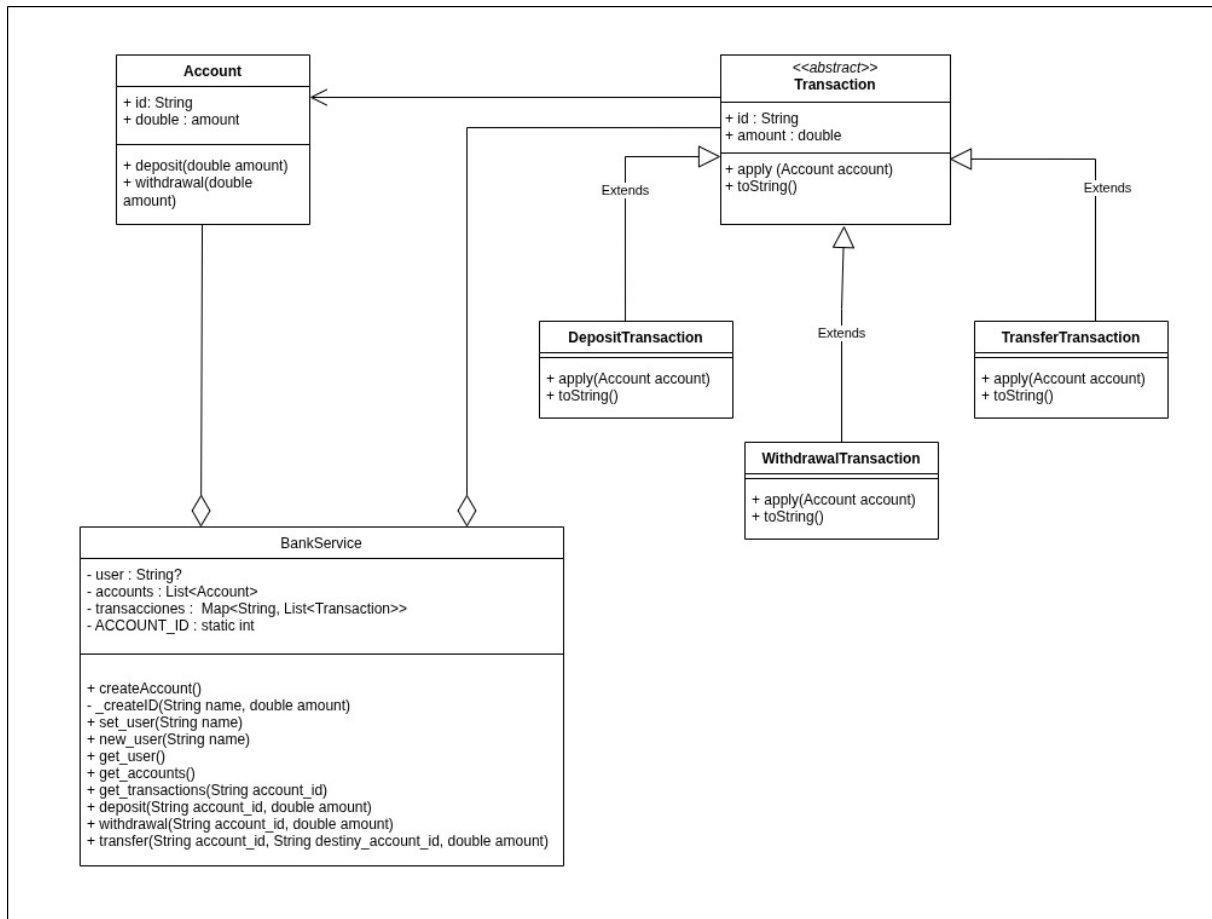


Figura 1: Diagrama UML

Del diagrama obtenemos que las entidades principales son *Account*, *Transaction*, y las que heredan de ella, y *BankService* y sus relaciones (*BankService* tiene una cuenta y una lista de transacciones). Ahora veremos detenidamente cada una de las clases asociadas a las entidades.

2.1.1. Clase Account

La clase *Account* representa una cuenta bancaria simple, con un identificador único de tipo `String` **id** y un saldo actual **amount** representado como un número decimal (`double`), el cual se inicializa en cero al crear la cuenta.

Los métodos que tiene son los siguientes:

- **bool deposit(double amount):** Realiza un depósito en la cuenta. Si el `amount` ingresado es mayor

que 0, se suma al saldo actual y el método retorna true. Si el amount es cero o negativo, no se realiza ninguna operación y el método retorna false.

- **bool withdrawal(double amount):** Realiza un retiro de la cuenta. El retiro se ejecuta únicamente si el amount es mayor que 0 y no excede el saldo disponible. En ese caso, se descuenta del saldo y el método retorna true. Si el amount no cumple estas condiciones, no se realiza el retiro y el método retorna false.

2.1.2. Clase Transaction

La clase Transaction es una clase base abstracta que define la estructura común de una transacción financiera. Cada transacción tiene un identificador único de tipo String **id** y un amount y debe implementar el método apply, que ejecuta la transacción sobre una cuenta dada.

Las clases que heredan son las siguientes:

- **DepositTransaction:** Al aplicar la transacción (apply), se realiza un depósito utilizando el método deposit de la clase Account. Si el depósito no es válido (por ejemplo, amount no positivo), lanza una excepción.
- **WithdrawalTransaction:** Usa el método withdrawal de Account para intentar retirar el amount especificado. Si el retiro no es válido (por amount negativo o fondos insuficientes), lanza una excepción.
- **TransferTransaction:** Esta clase tiene un atributo adicional que sería la cuenta de destino. El motivo de esta decisión es mantener el método apply sea uniforme en todas las clases. Al aplicar la transacción, intenta retirar el amount desde la cuenta origen. Si el retiro es exitoso, intenta depositar el mismo amount en la cuenta destino. Si el depósito en la cuenta destino falla, revierte el retiro original (depositando de nuevo en la cuenta origen) y lanza una excepción. Si el retiro inicial falla, también lanza una excepción.

2.1.3. Clase BankService

La clase BankService proporciona una capa de servicios bancarios básicos que permiten gestionar usuarios, cuentas y la ejecución de depósitos, retiros y transferencias, además de llevar un registro de transacciones por cuenta.

Los métodos que tiene son los siguientes:

- **void createAccount():** Crea una nueva cuenta asociada al usuario actual. Genera un ID único utilizando el nombre del usuario. Lanza una excepción si no hay un usuario establecido.
- **String _createID(String name, double amount):** Genera un identificador único para cuentas o transacciones. Si el amount es negativo, crea un ID usando solo el nombre y el contador; si es positivo, incluye el amount como parte del ID.
- **void new_user(String name):** Establece un nuevo usuario y crea una cuenta asociada. Si ya había un usuario, limpia las cuentas y transacciones anteriores antes de crear la nueva cuenta.
- **List<Map<String, dynamic>> get_transactions(String account_id):** Devuelve un historial de transacciones para una cuenta dada. Cada elemento del historial es un mapa que contiene el amount de la cuenta en el momento de la transacción y la transacción misma. El historial se reconstruye de forma inversa para reflejar correctamente los cambios en el saldo.
- **void deposit(String account_id, double amount):** Realiza un depósito a la cuenta especificada. Crea una transacción de tipo *DepositTransaction*, la aplica a la cuenta y la almacena en el historial. Lanza una excepción si no hay usuario activo.

- **`void withdrawal(String account_id, double amount):`** Realiza un retiro desde la cuenta especificada. Crea una transacción de tipo *WithdrawalTransaction*, la aplica a la cuenta y la guarda en el historial. Lanza una excepción si no hay usuario activo.
- **`void transfer(String account_id, String destiny_account_id, double amount):`** Ejecuta una transferencia de fondos desde una cuenta origen hacia una cuenta destino. Genera una transacción de tipo *TransferTransaction*, la aplica a ambas cuentas y actualiza sus historiales. Lanza excepciones si alguna de las cuentas no se encuentra.

2.2. Estructura de ficheros

La estructura de ficheros es muy sencilla, dentro de la carpeta lib encontramos el directorio model que contiene los archivos con las clases que modelan la lógica de la entidad bancaria, las cuentas y las transacciones entre ellas. Además tenemos el archivo main.dart que tiene toda la interfaz de la aplicación.

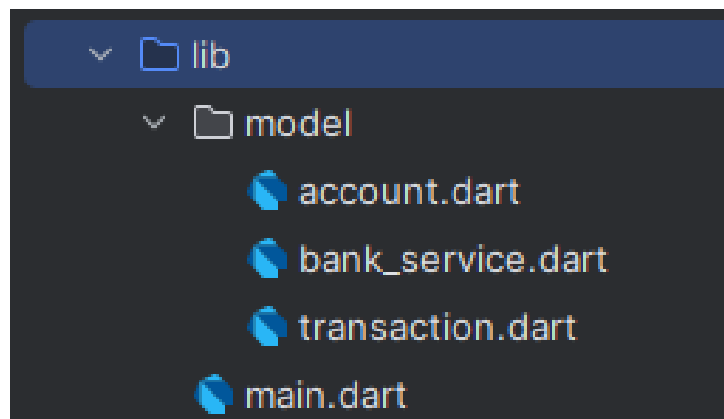


Figura 2: Estructura de ficheros

3. Diseño e implementación de pruebas

El objetivo principal es verificar el correcto funcionamiento de las clases *Account*, *Transaction* y *BankService* mediante la implementación de pruebas unitarias automatizadas.

3.1. Enfoque de Pruebas

Para garantizar la calidad del código y el cumplimiento de los requisitos funcionales, se ha adoptado un enfoque sistemático en la implementación de las pruebas unitarias. Este enfoque se basa en:

- Identificación del objetivo de la prueba
- Definición de condiciones de prueba específicas
- Planificación de datos de prueba representativos

Se implementaron un total de 12 pruebas, agrupadas en tres secciones principales, correspondientes a las tres clases fundamentales del sistema. Todas las pruebas realizadas han sido ejecutadas y han resultado exitosas, confirmando el correcto funcionamiento del código.

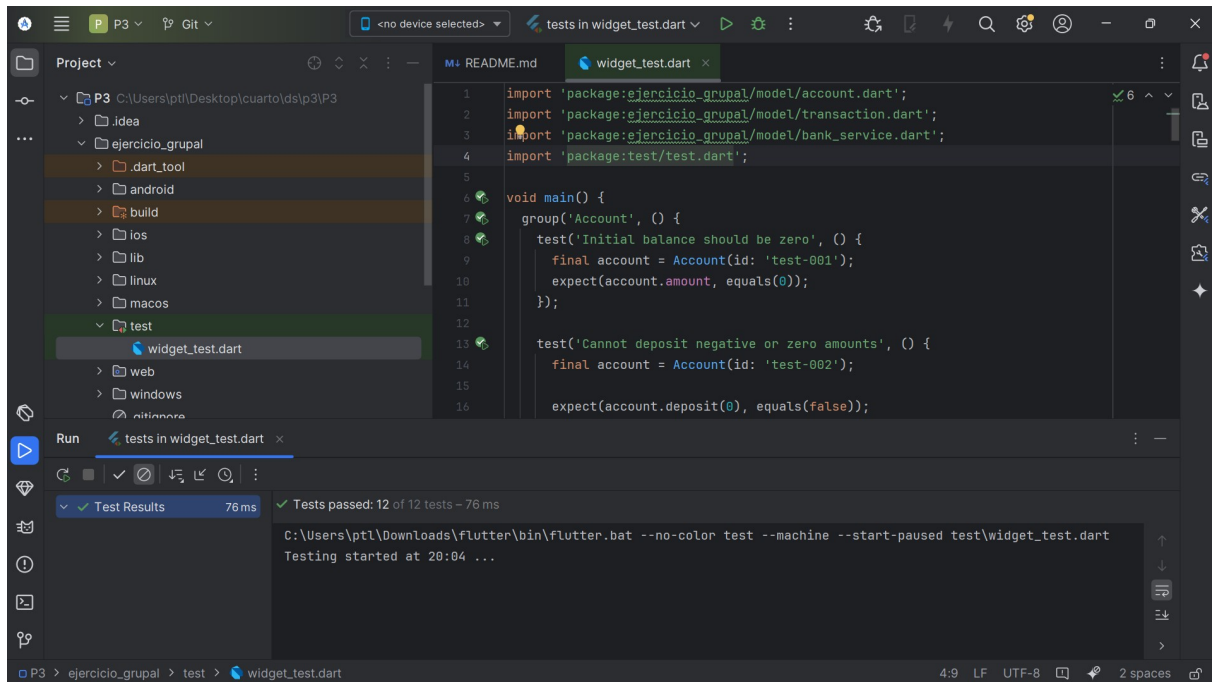


Figura 3: Ejecución del fichero de test

3.2. Herramientas utilizadas

Para la implementación de las pruebas unitarias se utilizó el paquete `test` de Dart, que ofrece una serie de funcionalidades para facilitar la escritura y ejecución de pruebas automatizadas.

3.3. Agrupaciones de tests

3.3.1. Grupo Account

Este grupo de pruebas se centra en verificar el comportamiento de la clase `Account`, que representa una cuenta bancaria en el sistema.

Prueba 1: El balance inicial de una cuenta debe ser cero

■ Objetivo:

- Verificar que cuando se crea una nueva cuenta, su saldo inicial es cero.

■ Justificación:

- Este test es fundamental para garantizar que todas las cuentas comiencen con un saldo inicial de cero.
- Evita posibles irregularidades en el sistema.
- Un saldo inicial incorrecto podría afectar todas las operaciones posteriores.

■ Condiciones de prueba:

- Crear una nueva instancia de `Account`
- Verificar que su propiedad `amount` sea exactamente cero

Prueba 2: No se permite depositar cantidades negativas o cero**■ Objetivo:**

- Comprobar que el método `deposit` rechaza cantidades negativas o cero.
- Verificar que solo acepta valores positivos.

■ Justificación:

- Esta validación es crucial para mantener la integridad del sistema bancario.
- Permitir depósitos negativos o cero podría causar comportamientos ilógicos.
- Podría permitir manipulaciones indebidas del saldo.

■ Condiciones de prueba:

- Intentar depositar una cantidad igual a cero
- Intentar depositar una cantidad negativa
- Depositar una cantidad positiva como caso válido
- Verificar que el saldo solo cambia en el caso válido

Prueba 3: No se permite retirar cantidades negativas o cero**■ Objetivo:**

- Verificar que el método `withdrawal` rechaza cantidades negativas o cero.
- Comprobar que solo acepta valores positivos.

■ Justificación:

- Esta validación es esencial para evitar operaciones sin sentido en el contexto bancario.
- Un retiro de cantidad cero o negativa no tiene sentido lógico.
- Podría introducir errores en la contabilidad.

■ Condiciones de prueba:

- Preparar una cuenta con saldo conocido
- Intentar retirar una cantidad igual a cero
- Intentar retirar una cantidad negativa
- Retirar una cantidad positiva como caso válido
- Verificar que el saldo solo cambia en el caso válido

3.3.2. Grupo Transaction

Este grupo de pruebas se enfoca en verificar el comportamiento de las diferentes clases de transacciones: `DepositTransaction`, `WithdrawalTransaction` y `TransferTransaction`.

Prueba 4: DepositTransaction.apply aumenta el saldo correctamente**■ Objetivo:**

- Comprobar que cuando se aplica una transacción de depósito a una cuenta, el saldo aumenta correctamente.

■ Justificación:

- Esta prueba es importante para garantizar que el mecanismo de aplicación de depósitos funcione correctamente.
- Es una operación fundamental en cualquier sistema bancario.

■ Condiciones de prueba:

- Crear una cuenta con saldo inicial cero
- Crear una transacción de depósito con una cantidad conocida
- Aplicar la transacción a la cuenta
- Verificar que el saldo de la cuenta ha aumentado exactamente en la cantidad especificada

Prueba 5: WithdrawalTransaction.apply lanza Exception cuando hay fondos insuficientes**■ Objetivo:**

- Verificar que cuando se intenta aplicar una transacción de retiro a una cuenta con saldo insuficiente, se lanza una excepción.

■ Justificación:

- Esta prueba es crucial para asegurar que el sistema no permita retirar más dinero del disponible.
- Mantiene la integridad de los saldos.

■ Condiciones de prueba:

- Crear una cuenta con un saldo conocido
- Crear una transacción de retiro con una cantidad mayor al saldo
- Verificar que al aplicar la transacción se lanza una excepción
- Comprobar que el saldo no ha sido modificado

Prueba 6: TransferTransaction.apply mueve fondos entre cuentas correctamente**■ Objetivo:**

- Comprobar que una transacción de transferencia mueve correctamente los fondos desde la cuenta de origen a la cuenta de destino.

■ Justificación:

- Las transferencias son operaciones críticas en cualquier sistema bancario.
- Es esencial verificar que los fondos se muevan correctamente entre cuentas.
- Previene pérdida o duplicación de dinero.

■ Condiciones de prueba:

- Crear dos cuentas: origen y destino
- Asignar un saldo conocido a la cuenta de origen
- Crear una transacción de transferencia
- Aplicar la transacción
- Verificar que el saldo de la cuenta origen ha disminuido correctamente
- Verificar que el saldo de la cuenta destino ha aumentado correctamente

3.3.3. Grupo BankService

Este grupo de pruebas se centra en verificar el comportamiento de la clase `BankService`, que proporciona la lógica de negocio principal para el sistema bancario.

Prueba 7: La lista inicial de cuentas está vacía

- **Objetivo:**
 - Comprobar que cuando se crea un nuevo servicio bancario, la lista de cuentas está inicialmente vacía.
- **Justificación:**
 - Verifica que el servicio bancario inicializa correctamente su estado interno.
 - Es crucial para el funcionamiento correcto de todas las operaciones posteriores.
- **Condiciones de prueba:**
 - Crear una nueva instancia de `BankService`
 - Verificar que la lista de cuentas está vacía

Prueba 8: deposit aumenta el saldo de la cuenta

- **Objetivo:**
 - Verificar que el método `deposit` del servicio bancario aumenta correctamente el saldo de la cuenta especificada.
- **Justificación:**
 - La funcionalidad de depósito es una operación básica en cualquier sistema bancario.
 - Es esencial que funcione correctamente a nivel de servicio.
- **Condiciones de prueba:**
 - Preparar el servicio bancario con un usuario y una cuenta
 - Registrar el saldo inicial de la cuenta
 - Realizar un depósito utilizando el servicio
 - Verificar que el saldo ha aumentado en la cantidad exacta depositada

Prueba 9: withdrawal lanza Exception cuando el saldo es insuficiente**■ Objetivo:**

- Comprobar que el método `withdrawal` del servicio bancario lanza una excepción cuando se intenta retirar más dinero del disponible.

■ Justificación:

- Es crucial que el sistema no permita retirar más dinero del disponible.
- Mantiene la integridad de los saldos.
- Evita números negativos no autorizados.

■ Condiciones de prueba:

- Preparar el servicio bancario con un usuario y una cuenta con saldo cero
- Intentar retirar una cantidad positiva
- Verificar que se lanza una excepción

Prueba 10: transfer mueve fondos correctamente**■ Objetivo:**

- Verificar que el método `transfer` del servicio bancario mueve correctamente los fondos entre dos cuentas.

■ Justificación:

- La transferencia de fondos es una operación crítica en cualquier sistema bancario.
- Es esencial verificar que los fondos se muevan correctamente entre cuentas a nivel de servicio.

■ Condiciones de prueba:

- Preparar el servicio bancario con un usuario y dos cuentas
- Depositar fondos en la cuenta de origen
- Realizar una transferencia entre las dos cuentas
- Verificar que los saldos de ambas cuentas se han actualizado correctamente

Prueba 11: transfer lanza Exception cuando los fondos son insuficientes**■ Objetivo:**

- Comprobar que el método `transfer` del servicio bancario lanza una excepción cuando se intenta transferir más dinero del disponible.

■ Justificación:

- Es crucial que el sistema no permita transferir más dinero del disponible.
- Mantiene la integridad de los saldos.
- Evita situaciones de inconsistencia.

■ Condiciones de prueba:

- Preparar el servicio bancario con un usuario y dos cuentas
- Intentar realizar una transferencia cuando la cuenta de origen no tiene fondos suficientes
- Verificar que se lanza una excepción

Prueba 12: txId genera identificadores únicos**■ Objetivo:**

- Verificar que el sistema genera identificadores de transacción únicos.
- Comprobar que son únicos incluso para operaciones realizadas en rápida sucesión.

■ Justificación:

- La unicidad de los identificadores de transacción es crítica para la trazabilidad.
- Garantiza el registro adecuado de las operaciones bancarias.
- Asegura que cada transacción sea identificable de manera inequívoca.

■ Condiciones de prueba:

- Preparar el servicio bancario con un usuario y una cuenta
- Realizar dos depósitos consecutivos
- Recuperar las transacciones asociadas a la cuenta
- Verificar que los identificadores de ambas transacciones son diferentes

4. Diseño de la interfaz

La interfaz gráfica, desarrollada en Flutter, tiene como objetivo proporcionar al usuario un entorno visual sencillo para que pueda interactuar con el sistema bancario simulado. Permite al usuario realizar una serie de operaciones:

- Registro de nuevas cuentas bancarias.
- Consulta del estado actual de la cuenta.
- Realización de depósitos y retiradas.
- Transferencias de fondos entre cuentas.

Todo esto se implementa mediante el uso de formularios y botones, con validaciones en tiempo real para el control de errores y mostrar mensajes informativos.

La interfaz se compone de:

- Un campo de texto para introducir el ID de la cuenta sobre la que operar.
- Campos para introducir importes, y en el caso de transferencias, un campo adicional para el ID de la cuenta que va a recibir el dinero.
- Botones para realizar las siguientes operaciones:
 - Crear cuenta
 - Depositar
 - Retirar
 - Transferir
 - Consultar cuenta

Cada botón realiza una acción sobre el objeto `BankService` que se encarga de gestionar las operaciones.

4.1. Control de errores y validaciones

Con el objetivo de garantizar la robustez y fiabilidad del sistema bancario, se implementan una serie de validaciones:

- **Saldo insuficiente:** Si el usuario intenta retirar más dinero del disponible en la cuenta, se lanza una excepción `StateError` y se muestra un mensaje de error en la interfaz.
- **Transferencias:**
 - Si se introduce un ID de cuenta inexistente como destino, se notifica el error correspondiente.
 - Si el saldo de la cuenta origen es insuficiente, no permite realizar la transferencia.
- **Importes inválidos:** No se permite realizar depósitos, retiradas o transferencias con cantidades negativas o cero. En estos casos, la interfaz actúa como si el botón no hubiera sido pulsado, de esta forma se impide ejecutar operaciones no válidas y obliga al usuario a corregir el importe antes de poder continuar.

4.2. Interacción

La interfaz implementada es reactiva, el texto del resultado de cada operación se actualiza dinámicamente en pantalla para ofrecer retroalimentación inmediata al usuario. Esto mejora la usabilidad, permitiendo detectar errores sin necesidad de revisar la consola.

Los widgets principales utilizados son:

- `TextField` para la entrada de datos.
- `ElevatedButton` para las acciones.
- `Text` para los mensajes de resultado.
- `setState()` para forzar la actualización al cambiar de estado tras una operación.

4.3. Resultado final

A continuación, se presentan varias capturas de pantalla que ilustran el funcionamiento de la interfaz.

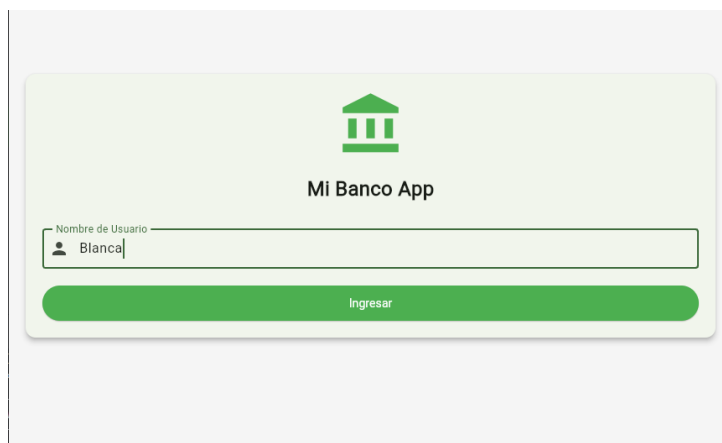


Figura 4: Inicio de sesión del sistema bancario

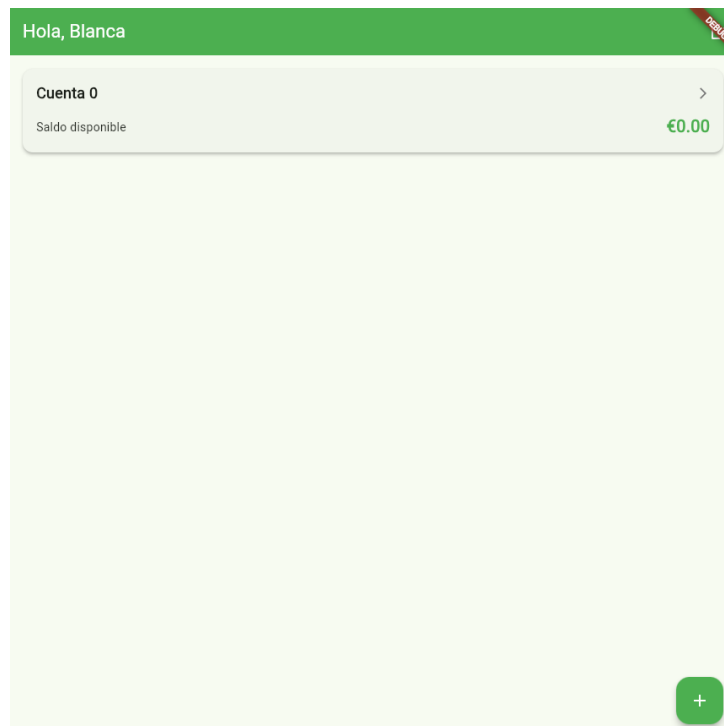


Figura 5: Vista de las cuentas del usuario

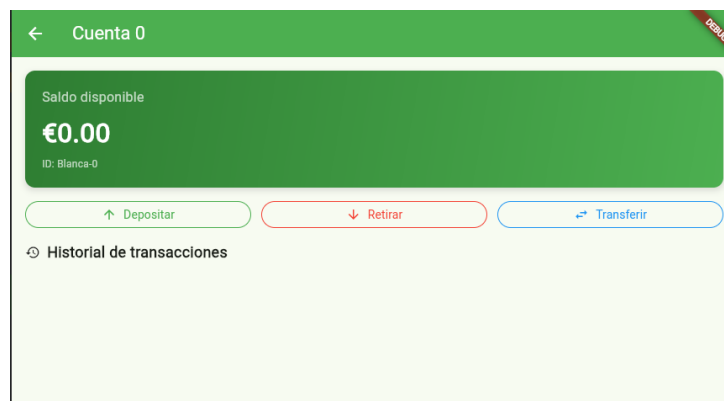


Figura 6: Vista inicial de la cuenta

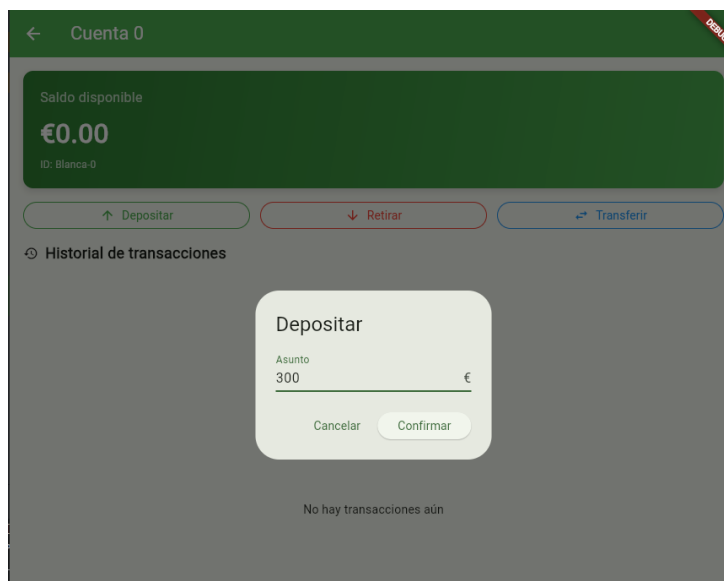


Figura 7: Realizar un depósito

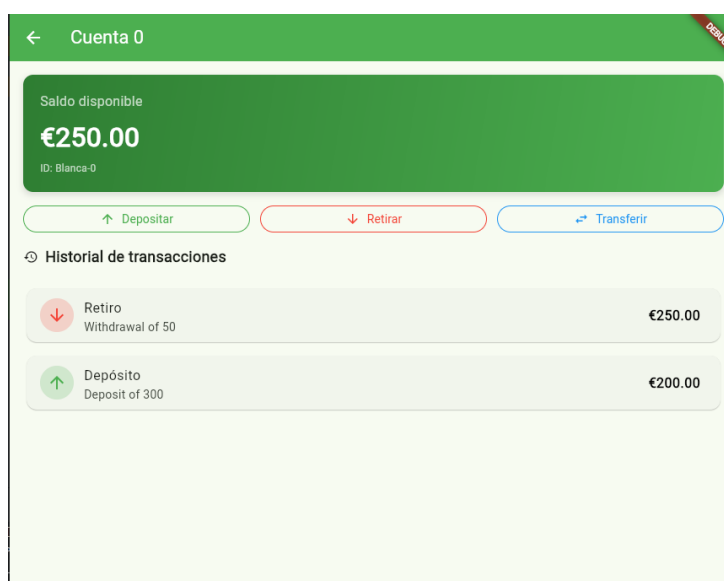


Figura 8: Resultado de retirar dinero

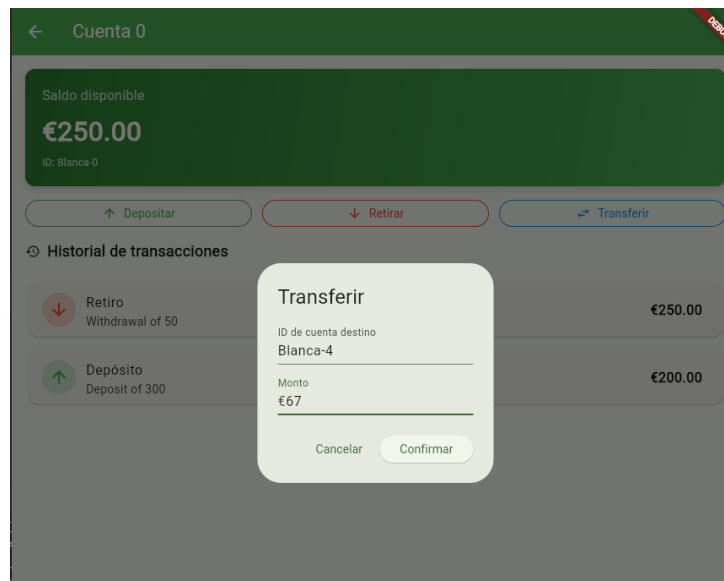


Figura 9: Transferir dinero a otra cuenta

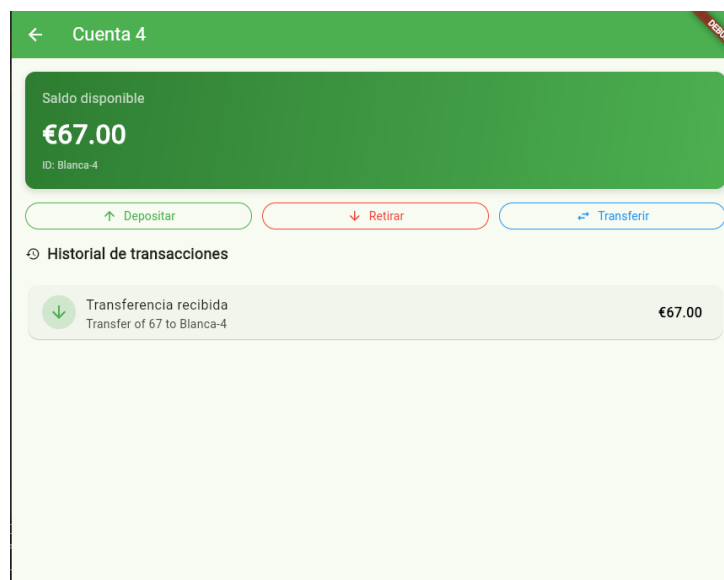


Figura 10: Resultado de la transferencia



Figura 11: Vista del historial de operaciones

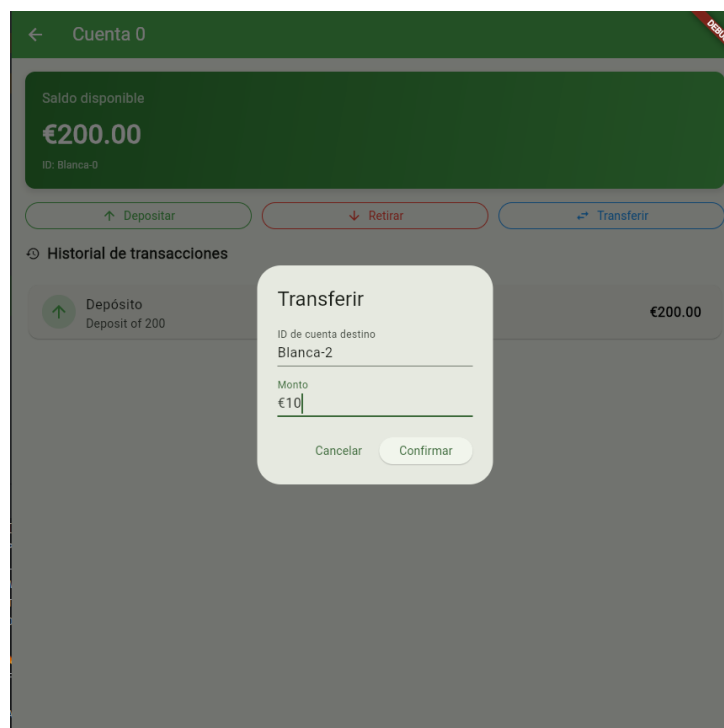


Figura 12: Intento de transferir a una cuenta con id inexistente

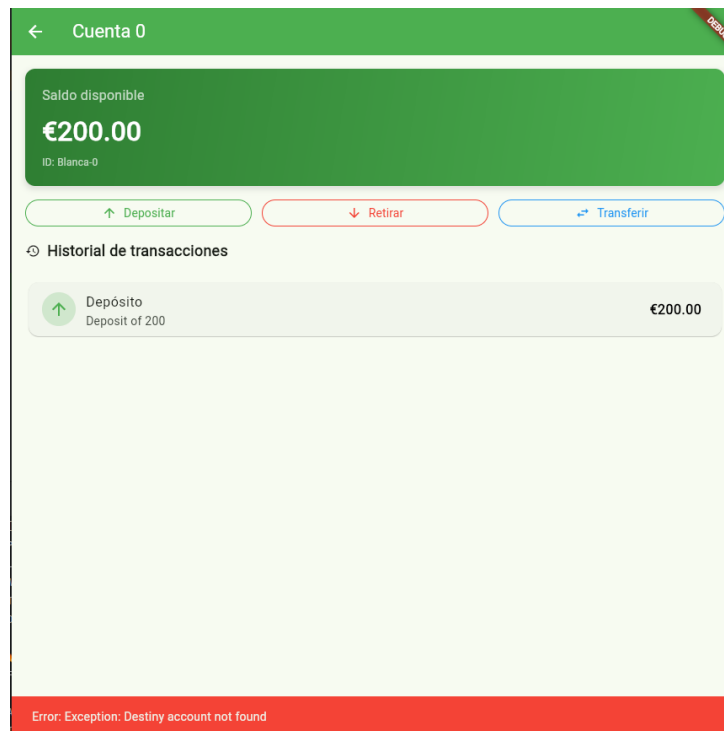


Figura 13: Error al intentar transferir a una cuenta con id inexistente

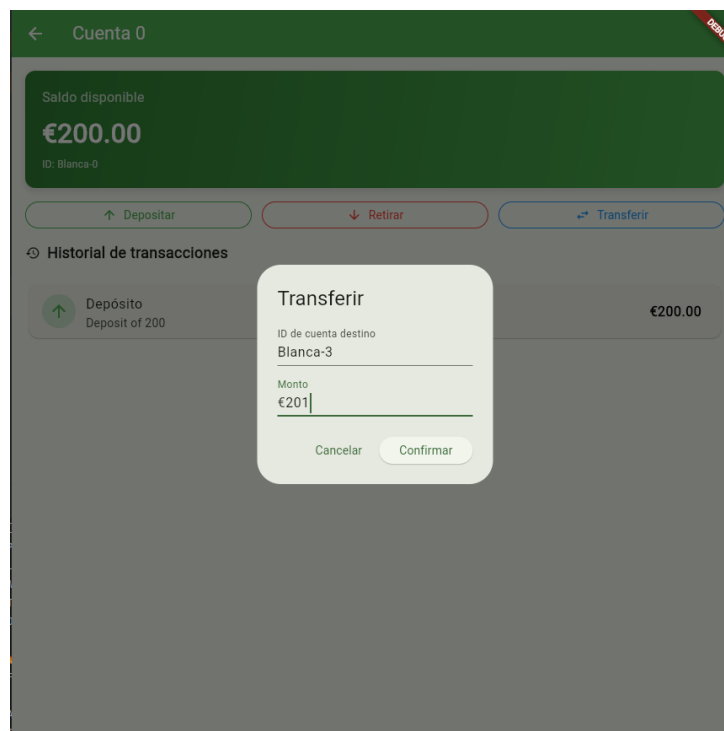


Figura 14: Intentar transferir más dinero del disponible

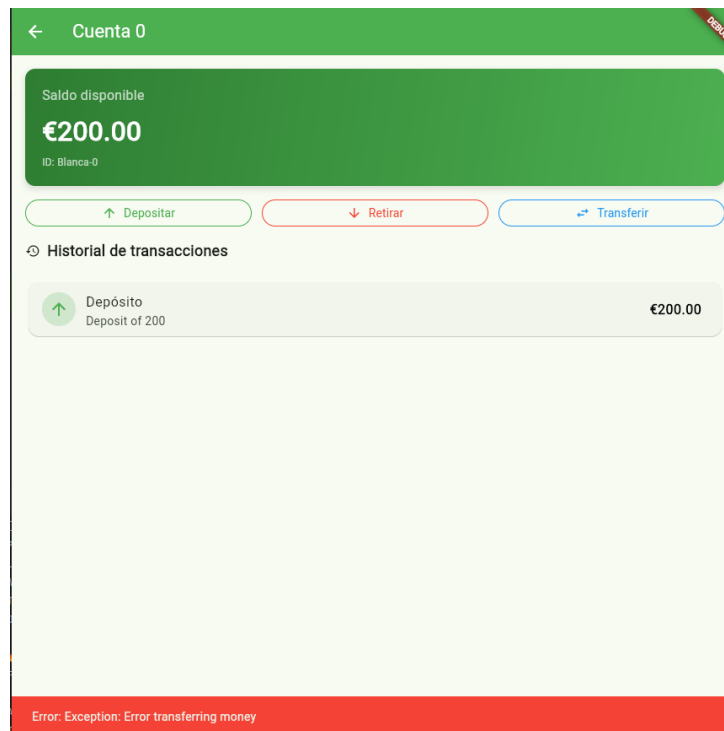


Figura 15: Error al intentar transferir más dinero del disponible

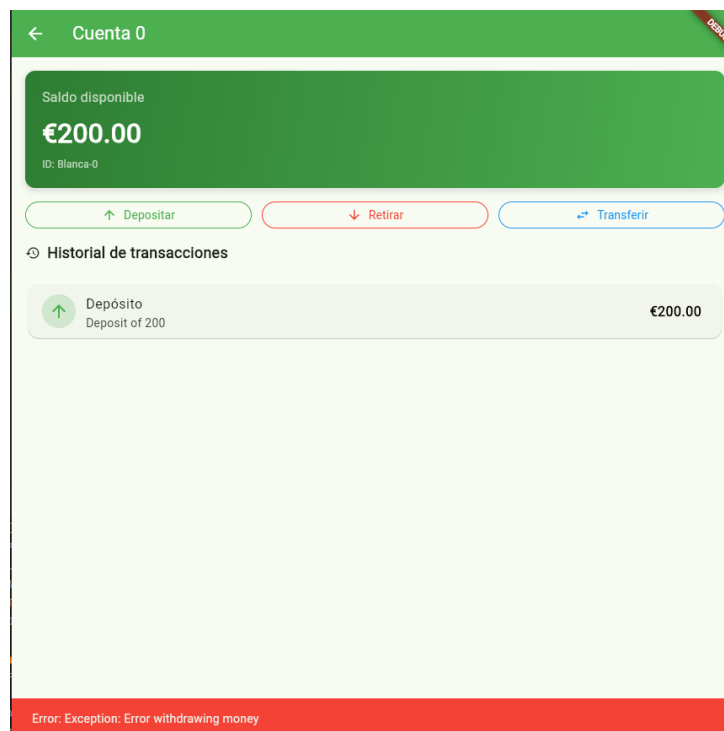


Figura 16: Error al intentar hacer un retiro inválido